

EDUCATIONAL REFERENCE

Graph Algorithms

Traversal, Pathfinding, Cycles, Components, Sorting & Trees

English

Russian / Русский

Armenian / Հայերեն

TOPICS COVERED

- Breadth-First Search (BFS)
- Dijkstra's Shortest Path Algorithm
- DFS for Cycle Detection
- Finding Connected Components
- Kahn's Algorithm (Topological Sort)
- Kruskal's Algorithm (MST)

Table of Contents

1. Breadth-First Search (BFS)	
2. Dijkstra's Algorithm — Shortest Path	
3. DFS for Cycle Detection	
4. Finding Connected Components	
5. Kahn's Algorithm — Topological Sorting	
6. Kruskal's Algorithm — Minimum Spanning Tree	

Breadth-First Search (BFS)

Category: Graph Traversal

ENGLISH

What is BFS?

Breadth-First Search (BFS) is a fundamental graph traversal algorithm that explores a graph level by level, starting from a chosen source vertex. It visits all neighbors of the current vertex before moving to the next level of vertices. BFS uses a **queue** (FIFO) data structure to manage the order of exploration.

How It Works

The algorithm begins by enqueueing the start vertex and marking it as visited. It then repeatedly dequeues the front vertex, processes it, and enqueues all of its unvisited neighbors. This process continues until the queue is empty.

1. Mark all vertices as unvisited.
2. Enqueue the start vertex; mark it visited.
3. While queue is not empty: dequeue vertex u , process it, enqueue all unvisited neighbors of u and mark them visited.

Pseudocode

```
BFS(Graph G, start vertex s):  
    create a queue Q  
    mark s as visited  
    enqueue s into Q  
    while Q is not empty:  
        u = dequeue(Q)  
        for each neighbor v of u:  
            if v is not visited:  
                mark v as visited  
                enqueue v into Q
```

Python Implementation

```

from collections import deque

def bfs(graph, start):
    visited = set()
    queue = deque([start])
    visited.add(start)
    order = []
    while queue:
        node = queue.popleft()
        order.append(node)
        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)
    return order

# Example
graph = {0:[1,2], 1:[0,3,4], 2:[0,5], 3:[1], 4:[1], 5:[2]}
print(bfs(graph, 0)) # [0, 1, 2, 3, 4, 5]

```

Applications

- Finding the **shortest path** in an unweighted graph.
- Level-order traversal of trees.
- Web crawling and social-network friend-suggestion systems.
- Detecting bipartiteness of a graph.
- Broadcasting in networks (e.g., peer-to-peer systems).

Complexity

Metric	Value	Note
Time complexity	$O(V + E)$	V = vertices, E = edges
Space complexity	$O(V)$	Queue + visited set

BFS guarantees the shortest path in **unweighted** graphs. It is not suitable for weighted graphs where edge costs differ — use Dijkstra's algorithm in that case.

Что такое BFS?

Поиск в ширину (BFS, Breadth-First Search) — это фундаментальный алгоритм обхода графа, который исследует граф уровень за уровнем, начиная с выбранной вершины-источника. Алгоритм посещает всех соседей текущей вершины, прежде чем перейти к следующему уровню. Для управления порядком обхода используется структура данных **очередь** (FIFO).

Как работает алгоритм

Алгоритм начинает с помещения стартовой вершины в очередь и пометки её как посещённой. Затем он многократно извлекает вершину из очереди, обрабатывает её и добавляет в очередь всех её непосещённых соседей. Процесс продолжается до тех пор, пока очередь не опустеет.

1. Пометить все вершины как непосещённые.
2. Добавить стартовую вершину в очередь и пометить её как посещённую.
3. Пока очередь не пуста: извлечь вершину u , обработать её, добавить в очередь всех непосещённых соседей u и пометить их как посещённые.

Реализация на Python

```
from collections import deque

def bfs(graph, start):
    visited = set()
    queue = deque([start])
    visited.add(start)
    order = []
    while queue:
        node = queue.popleft()
        order.append(node)
        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)
    return order
```

Применения

- Нахождение **кратчайшего пути** в невзвешенном графе.
- Обход дерева по уровням.
- Веб-краулинг и системы рекомендации друзей в социальных сетях.
- Определение двудольности графа.

- Широковещательная рассылка в сетях.

Сложность алгоритма

Метрика	Значение	Примечание
Временная сложность	$O(V + E)$	V — вершины, E — рёбра
Пространственная сложность	$O(V)$	Очередь + множество посещённых

BFS гарантирует нахождение кратчайшего пути в **невзвешенных** графах. Для взвешенных графов следует использовать алгоритм Дейкстры.

ՀԱՅԵՐԵՆ

Ի՞նչ է BFS-ը

Լայնությամբ որոնումը (BFS — Breadth-First Search) գրաֆի հատման հիմնական ալգորիթմ է, որն ուսումնասիրում է գրաֆը մակարդակ առ մակարդակ՝ սկսելով ընտրված սկզբնաղբյուր գագաթից: Ալգորիթմն անցնում է ընթացիկ գագաթի բոլոր հարևաններով, մինչ անցնի հաջորդ մակարդակ: Հերթի (FIFO) տվյալների կառուցվածքն օգտագործվում է հատման կարգը կառավարելու համար:

Ինչպես է աշխատում

Ալգորիթմն սկսում է սկզբնական գագաթը հերթ տեղադրելով և նշելով այն որպես այցելված: Ապա հաջորդաբար հերթից հանում է գագաթ, մշակում է այն և ավելացնում է բոլոր չայցելված հարևաններին հերթում: Գործընթացը շարունակվում է մինչ հերթը դատարկ դառնա:

1. Բոլոր գագաթները նշել որպես չայցելված:
2. Սկզբնական գագաթն ավելացնել հերթ, նշել այցելված:
3. Քանի դեռ հերթը դատարկ չէ. հերթից հանել u գագաթ, մշակել, ավելացնել բոլոր չայցելված հարևաններ:

Python իրականացում

```
from collections import deque

def bfs(graph, start):
    visited = set()
    queue = deque([start])
    visited.add(start)
    order = []
    while queue:
        node = queue.popleft()
        order.append(node)
        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)
    return order
```

Կիրառություններ

- Կշռվածք չունեցող գրաֆում **ամենակարճ ճանապարհի** գտնում:
- Ծառի մակարդակ-առ-մակարդակ հատում:
- Վեր-ինդեքսավորում և ընկերների առաջարկ սոցիալական ցանցերում:
- Գրաֆի երկու-կողմանիության ստուգում:
- Ցանցերում հեռարձակում:

Բարդություն

Չափանիշ	Արժեք	Նշում
Ժամանակային բարդություն	$O(V + E)$	V — գագաթներ, E — կողեր
Հիշողության բարդություն	$O(V)$	Հերթ + այցելվածների բազմություն

BFS-ն երաշխավորում է ամենակարճ ճանապարհ **կշռվածք չունեցող** գրաֆներում:
Կշռված գրաֆների համար անհրաժեշտ է օգտագործել Դեյկստրայի ալգորիթմը:

Dijkstra's Algorithm

Category: *Shortest Path*

ENGLISH

What is Dijkstra's Algorithm?

Dijkstra's algorithm, proposed by Edsger W. Dijkstra in 1956, finds the **shortest path** from a single source vertex to all other vertices in a graph with **non-negative** edge weights. It is one of the most widely used algorithms in computer science, powering GPS navigation, network routing protocols (OSPF), and map applications.

Core Idea

The algorithm maintains a set of vertices whose shortest distances from the source are already finalized. At each step it picks the unvisited vertex with the smallest known distance (greedy choice), finalizes its distance, and *relaxes* all edges leaving that vertex — i.e., it updates the distance to each neighbor if a shorter path is found via the current vertex.

Step-by-Step

1. Initialize all distances to infinity except the source (distance = 0).
2. Insert the source into a min-priority queue with priority 0.
3. While the priority queue is not empty:
 - Extract the vertex u with the minimum distance.
 - For each neighbor v of u : if $\text{dist}[u] + \text{weight}(u,v) < \text{dist}[v]$, update $\text{dist}[v]$ and push v into the queue.
4. Return the dist array.

Python Implementation (Min-Heap)


```

import heapq

def dijkstra(graph, src):
    # graph: {u: [(v, weight), ...]}
    dist = {node: float('inf') for node in graph}
    dist[src] = 0
    heap = [(0, src)]    # (distance, vertex)
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist[u]:
            continue
        for v, w in graph[u]:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                heapq.heappush(heap, (dist[v], v))
    return dist

# Example
graph = {
    'A': [('B', 4), ('C', 1)],
    'B': [('C', 2), ('D', 5)],
    'C': [('B', 1), ('D', 8)],
    'D': []
}
print(dijkstra(graph, 'A'))
# {'A': 0, 'B': 2, 'C': 1, 'D': 7}

```

Why Non-Negative Weights?

The algorithm relies on the invariant that once a vertex is finalized, its distance cannot decrease further. Negative edges can break this invariant — use the Bellman-Ford algorithm for graphs with negative weights.

Applications

- GPS and mapping services (Google Maps, OpenStreetMap).
- Network routing (OSPF protocol in IP networks).
- Game AI pathfinding (combined with heuristics in A*).
- Flight and transportation route optimization.

Complexity

Implementation	Time Complexity	Space
Simple array	$O(V^2)$	$O(V)$

Binary min-heap	$O((V + E) \log V)$	$O(V + E)$
Fibonacci heap	$O(E + V \log V)$	$O(V + E)$

РУССКИЙ

Что такое алгоритм Дейкстры?

Алгоритм Дейкстры, предложенный Эдсгером Дейкстрой в 1956 году, находит **кратчайший путь** от одной вершины-источника до всех остальных вершин в графе с **неотрицательными** весами рёбер. Это один из наиболее широко используемых алгоритмов в информатике, применяемый в GPS-навигации, сетевых протоколах маршрутизации (OSPF) и картографических приложениях.

Основная идея

Алгоритм поддерживает множество вершин, для которых кратчайшее расстояние от источника уже установлено. На каждом шаге выбирается непосещённая вершина с наименьшим известным расстоянием (жадный выбор), её расстояние фиксируется, и выполняется *релаксация* всех выходящих рёбер — то есть обновляются расстояния до соседей, если через текущую вершину обнаружен более короткий путь.

Пошаговое описание

1. Инициализировать расстояния до всех вершин бесконечностью, кроме источника (расстояние = 0).
2. Добавить источник в приоритетную очередь с приоритетом 0.
3. Пока очередь не пуста: извлечь вершину с минимальным расстоянием; для каждого соседа проверить, нельзя ли улучшить путь.
4. Вернуть массив расстояний.

Реализация на Python

```
import heapq

def dijkstra(graph, src):
    dist = {node: float('inf') for node in graph}
    dist[src] = 0
    heap = [(0, src)]
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist[u]:
            continue
        for v, w in graph[u]:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                heapq.heappush(heap, (dist[v], v))
    return dist
```

Почему только неотрицательные веса?

Алгоритм основан на инварианте: после фиксации расстояния до вершины оно не может уменьшиться. Отрицательные рёбра нарушают этот инвариант. Для графов с отрицательными весами используйте алгоритм Беллмана-Форда.

Применения

- GPS и картографические сервисы (Google Maps, OpenStreetMap).
- Сетевая маршрутизация (протокол OSPF).
- Поиск пути в игровом ИИ (в сочетании с эвристиками — алгоритм A*).
- Оптимизация транспортных маршрутов.

Сложность

Реализация	Временная сложность	Память
Простой массив	$O(V^2)$	$O(V)$
Двоичная куча	$O((V + E) \log V)$	$O(V + E)$
Куча Фибоначчи	$O(E + V \log V)$	$O(V + E)$

Ի՞նչ է Դեյկստրայի ալգորիթմը

Դեյկստրայի ալգորիթմը, որն առաջարկվել է Էդսգեր Դեյկստրայի կողմից 1956 թ.-ին, գտնում է **ամենակարճ ճանապարհ** մեկ սկզբնաղբյուր գագաթից մինչ բոլոր մյուս գագաթներ՝ **ոչ-բացասական** կշիռ ունեցող կողերով գրաֆում: Սա ամենաշատ օգտագործվող ալգորիթմներից մեկն է informatika-ում:

Հիմնական գաղափար

Ալգորիթմն ամեն քայլի ընտրում է ամենամոտ (ամենացածր հեռավորությամբ) հայտնի չայցելված գագաթը (ազահ ընտրություն), ամրագրում է դրա հեռավորությունը, ապա *թուլացնում* (relaxes) բոլոր ելնող կողերը: Եթե հարևան գագաթ ավելի կարճ ճանապարհ է հայտնաբերվում ընթացիկ գագաթի միջոցով, հեռավորությունը թարմացվում է:

Python իրականացում

```
import heapq

def dijkstra(graph, src):
    dist = {node: float('inf') for node in graph}
    dist[src] = 0
    heap = [(0, src)]
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist[u]:
            continue
        for v, w in graph[u]:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                heapq.heappush(heap, (dist[v], v))
    return dist
```

Կիրառություններ

- GPS և քարտեզագրման ծառայություններ (Google Maps, OpenStreetMap):
- Ցանցային երթուղի (OSPF արձանագրություն):
- Խաղային ԱԻ-ի ուղի-որոնում (A* ալգորիթմ):
- Փոխադրության երթուղու օպտիմալացում:

Բարդություն

Պարզ զանգված	$O(V^2)$	$O(V)$
Երկուսական կույտ	$O((V + E) \log V)$	$O(V + E)$
Ֆիբոնաչիի կույտ	$O(E + V \log V)$	$O(V + E)$

Ալգորիթմը կիրառելի է **միայն ոչ-բացասական** կշիռ ունեցող կողերի դեպքում:
Բացասական կշիռների համար օգտագործեք Բելլման-Ֆորդի ալգորիթմը:

DFS for Cycle Detection

Category: *Cycle Detection*

ENGLISH

What is Cycle Detection?

A **cycle** in a graph is a path that starts and ends at the same vertex without repeating any edge or vertex. Detecting cycles is critical in many applications: dependency resolution, deadlock detection in operating systems, validating circuit designs, and detecting infinite loops in program analysis.

Depth-First Search (DFS) naturally reveals cycles by tracking a *recursion stack* (for directed graphs) or checking if a visited neighbor is not the parent (for undirected graphs).

Directed Graph — Recursion Stack Method

During DFS, maintain two boolean arrays: `visited[]` and `rec_stack[]`. A cycle exists if DFS reaches a vertex that is already in the current recursion stack.

```
def has_cycle_directed(graph):
    visited = set()
    rec_stack = set()

    def dfs(v):
        visited.add(v)
        rec_stack.add(v)
        for neighbor in graph.get(v, []):
            if neighbor not in visited:
                if dfs(neighbor):
                    return True
            elif neighbor in rec_stack:
                return True # back edge => cycle found
        rec_stack.remove(v)
        return False

    for node in graph:
        if node not in visited:
            if dfs(node):
                return True
    return False

# Example
graph = {'A': ['B'], 'B': ['C'], 'C': ['A']}
print(has_cycle_directed(graph)) # True
```

Undirected Graph — Parent Tracking

For undirected graphs, a cycle exists when a visited neighbor is not the immediate parent of the current vertex during DFS.

```
def has_cycle_undirected(graph):
    visited = set()

    def dfs(v, parent):
        visited.add(v)
        for neighbor in graph.get(v, []):
            if neighbor not in visited:
                if dfs(neighbor, v):
                    return True
            elif neighbor != parent:
                return True # cycle found
        return False

    for node in graph:
        if node not in visited:
            if dfs(node, -1):
                return True
    return False
```

Key Concepts

- **Back edge:** an edge from a vertex to one of its ancestors in the DFS tree — indicates a cycle in a directed graph.
- **Cross edge / forward edge:** do not indicate a cycle in directed graphs.
- In undirected graphs, any edge to a visited non-parent vertex means a cycle.

Applications

- Deadlock detection in operating systems.
- Dependency graphs (package managers, build systems).
- Detecting loops in linked lists and program flow graphs.
- Checking if a graph is a valid DAG (required by topological sort).

Complexity

Metric	Value
Time complexity	$O(V + E)$
Space complexity	$O(V)$ — recursion stack

Что такое обнаружение цикла?

Цикл в графе — это путь, начинающийся и заканчивающийся в одной и той же вершине, не повторяя ни одного ребра или вершины. Обнаружение циклов важно во многих задачах: разрешение зависимостей, обнаружение взаимоблокировок (deadlock) в ОС, валидация схем и анализ программного кода.

Поиск в глубину (DFS) естественным образом обнаруживает циклы путём отслеживания *стека рекурсии* (для ориентированных графов) или проверки посещённых соседей (для неориентированных).

Ориентированный граф — метод стека рекурсии

В процессе DFS поддерживаются два массива: `visited[]` и `rec_stack[]`. Цикл обнаружен, если DFS достигает вершины, уже находящейся в текущем стеке рекурсии (обратное ребро).

```
def has_cycle_directed(graph):
    visited = set()
    rec_stack = set()

    def dfs(v):
        visited.add(v)
        rec_stack.add(v)
        for neighbor in graph.get(v, []):
            if neighbor not in visited:
                if dfs(neighbor):
                    return True
            elif neighbor in rec_stack:
                return True # обратное ребро => цикл найден
        rec_stack.remove(v)
        return False

    for node in graph:
        if node not in visited:
            if dfs(node):
                return True
    return False
```

Неориентированный граф — отслеживание родителя

Для неориентированных графов цикл существует, когда посещённый сосед не является непосредственным родителем текущей вершины в DFS-дереве.

Применения

- Обнаружение взаимоблокировок в операционных системах.
- Графы зависимостей (менеджеры пакетов, системы сборки).
- Обнаружение петель в связных списках и графах потоков программ.
- Проверка, является ли граф корректным DAG.

Сложность

Метрика	Значение
Временная сложность	$O(V + E)$
Пространственная сложность	$O(V)$ — стек рекурсии



Ի՞նչ է ցիկլի հայտնաբերումը

Գրաֆում **ցիկլ** նշանակում է ճանապարհ, որը սկսվում և վերջանում է նույն գագաթով՝ չկրկնելով ոչ մի կող կամ գագաթ: Ցիկլերի հայտնաբերումն անհրաժեշտ է կախվածությունների լուծման, ՕՀ-ում deadlock-ի հայտնաբերման, սխեմաների ստուգման և ծրագրերի վերլուծության համար:

Խորությամբ որոնումը (DFS) բնականաբար բացահայտում է ցիկլերը՝ հետևելով *ռեկուրսիայի steak-ին* (ուղղորդված գրաֆների համար) կամ ստուգելով՝ արդյոք այցելված հարևանը ծնող չէ (ոչ ուղղորդված գրաֆների համար):

Python իրականացում (ուղղորդված գրաֆ)

```
def has_cycle_directed(graph):
    visited = set()
    rec_stack = set()

    def dfs(v):
        visited.add(v)
        rec_stack.add(v)
        for neighbor in graph.get(v, []):
            if neighbor not in visited:
                if dfs(neighbor):
                    return True
            elif neighbor in rec_stack:
                return True # հետ կող => ցիկլ հայտնաբերված
        rec_stack.remove(v)
        return False

    for node in graph:
        if node not in visited:
            if dfs(node):
                return True
    return False
```

Կիրառություններ

- Deadlock-ի հայտնաբերում օպերացիոն համակարգերում:
- Կախվածության գրաֆներ (փաթեթների կառավարիչներ, կառուցման համակարգեր):
- Կապված ցուցակներում օղակների հայտնաբերում:
- DAG-ի վավերականության ստուգում (տոպոլոգիական դասավորության նախապայման):

Բարդություն

Չափանիշ	Արժեք
Ժամանակային բարդություն	$O(V + E)$
Հիշողության բարդություն	$O(V)$ — ռեկուրսիայի stack

Finding Connected Components

Category: *Connected Components*

ENGLISH

What are Connected Components?

A **connected component** of an undirected graph is a maximal subgraph in which every pair of vertices is connected by a path. In other words, you can reach any vertex from any other vertex within the component, but you cannot reach vertices in a different component.

For **directed graphs**, we distinguish *weakly connected components* (connected when edge directions are ignored) and *strongly connected components* (SCC — every vertex is reachable from every other within the SCC). Kosaraju's and Tarjan's algorithms solve the SCC problem.

Algorithm — DFS / BFS Labeling

The standard approach iterates over all vertices. For each unvisited vertex, a DFS (or BFS) is launched, and all vertices reachable from it are assigned the same component label. The number of launches equals the number of connected components.

```

def find_components(graph):
    visited = {}
    component_id = 0

    def dfs(v, comp):
        visited[v] = comp
        for neighbor in graph.get(v, []):
            if neighbor not in visited:
                dfs(neighbor, comp)

    for node in graph:
        if node not in visited:
            dfs(node, component_id)
            component_id += 1

    return visited, component_id

# Example
graph = {0:[1,2], 1:[0,2], 2:[0,1], 3:[4], 4:[3], 5:[]}
labels, count = find_components(graph)
print(f"Number of components: {count}")
# Component 0: {0,1,2} Component 1: {3,4} Component 2: {5}

```

Union-Find (Disjoint Set Union) Approach

An alternative and often more efficient approach uses the Union-Find (DSU) data structure. Each edge merges two sets, and the number of remaining distinct roots equals the number of components.

```

class UnionFind:
    def __init__(self, n):
        self.parent = list(range(n))
        self.rank = [0] * n

    def find(self, x):
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])
        return self.parent[x]

    def union(self, x, y):
        rx, ry = self.find(x), self.find(y)
        if rx == ry:
            return
        if self.rank[rx] < self.rank[ry]:
            rx, ry = ry, rx
        self.parent[ry] = rx
        if self.rank[rx] == self.rank[ry]:
            self.rank[rx] += 1

def count_components(n, edges):
    uf = UnionFind(n)
    for u, v in edges:
        uf.union(u, v)
    return len(set(uf.find(i) for i in range(n)))

```

Applications

- Network analysis: finding isolated sub-networks.
- Image processing: blob detection and segmentation.
- Social network analysis: finding isolated communities.
- Determining reachability in infrastructure graphs (power grids, roads).

Complexity

Method	Time	Space
DFS / BFS	$O(V + E)$	$O(V)$
Union-Find (path compression + union by rank)	$O(E \cdot \alpha(V)) \approx O(E)$	$O(V)$

Что такое связные компоненты?

Связная компонента неориентированного графа — это максимальный подграф, в котором каждая пара вершин соединена путём. Внутри компоненты можно добраться из любой вершины до любой другой, а между компонентами — нет.

Для **ориентированных графов** различают *слабосвязные компоненты* (связь без учёта направлений) и *сильносвязные компоненты* (SCC) — для них применяются алгоритмы Косарайю и Тарьяна.

Алгоритм — разметка через DFS/BFS

Стандартный подход перебирает все непосещённые вершины. Из каждой запускается DFS/BFS, все достижимые вершины получают один номер компоненты. Количество запусков равно числу связных компонент.

```
def find_components(graph):
    visited = {}
    component_id = 0

    def dfs(v, comp):
        visited[v] = comp
        for neighbor in graph.get(v, []):
            if neighbor not in visited:
                dfs(neighbor, comp)

    for node in graph:
        if node not in visited:
            dfs(node, component_id)
            component_id += 1

    return visited, component_id
```

Применения

- Анализ сетей: поиск изолированных подсетей.
- Обработка изображений: обнаружение и сегментация объектов.
- Анализ социальных сетей: поиск изолированных сообществ.
- Анализ инфраструктурных графов (электросети, дороги).

Сложность

Метод	Время	Память
DFS / BFS	$O(V + E)$	$O(V)$

ՀԱՅԵՐԵՆ

Ի՞նչ են կապակցված բաղադրիչները

Ոչ ուղղորդված գրաֆի **կապակցված բաղադրիչ**ն այն առավելագույն ենթագրաֆն է, որտեղ ցանկացած գագաթ-գույգ կապված է ճանապարհով: Բաղադրիչի ներսում ցանկացած գագաթից կարելի է հասնել ցանկացած մյուս գագաթ, մինչև տարբեր բաղադրիչների միջև — ոչ:

Ալգորիթմ — DFS/BFS-ով նշագրում

Ստանդարտ մոտեցումը կրկնում է բոլոր չայցելված գագաթների վրա: Ամեն անգամ DFS/BFS-ն գործարկվում է, և բոլոր հասանելի գագաթները ստանում են նույն բաղադրիչ-համարը: Գործարկումների քանակը հավասար է կապակցված բաղադրիչների քանակին:

```
def find_components(graph):
    visited = {}
    component_id = 0

    def dfs(v, comp):
        visited[v] = comp
        for neighbor in graph.get(v, []):
            if neighbor not in visited:
                dfs(neighbor, comp)

    for node in graph:
        if node not in visited:
            dfs(node, component_id)
            component_id += 1

    return visited, component_id
```

Կիրառություններ

- Ցանցի վերլուծություն. մեկուսի ենթացանցերի գտնում:
- Պատկերի մշակում. բծերի հայտնաբերում և հատվածավորում:
- Սոցիալական ցանցերի վերլուծություն. մեկուսի համայնքների գտնում:

- Ենթակառուցվածքային գրաֆների հասանելիության վերլուծություն:

Բարդություն

Մեթոդ	Ժամանակ	Հիշողություն
DFS / BFS	$O(V + E)$	$O(V)$
Union-Find (ճանապարհի սեղմում + կարգ)	$O(E \cdot \alpha(V)) \approx O(E)$	$O(V)$

Kahn's Algorithm

Category: Topological Sorting

ENGLISH

What is Topological Sorting?

A **topological sort** of a Directed Acyclic Graph (DAG) is a linear ordering of its vertices such that for every directed edge (u, v) , vertex u appears before vertex v in the ordering. Topological sorting is only possible in a DAG — a graph with no directed cycles.

Kahn's Algorithm achieves topological sorting using an iterative, BFS-based approach by repeatedly removing vertices with in-degree zero.

In-degree Concept

The **in-degree** of a vertex is the number of directed edges pointing into it. A vertex with in-degree 0 has no prerequisites — it can be processed first.

Step-by-Step Algorithm

1. Compute the in-degree of every vertex.
2. Add all vertices with in-degree 0 to a queue.
3. While the queue is not empty:
 - Dequeue a vertex u and append it to the result list.
 - For each neighbor v of u : decrement in-degree of v . If it becomes 0, enqueue v .
4. If result list length $< V$, the graph contains a cycle (no valid topological order).

Python Implementation

```

from collections import deque

def kahn_topological_sort(graph, n):
    # graph: {u: [v1, v2, ...]}
    in_degree = [0] * n
    for u in graph:
        for v in graph[u]:
            in_degree[v] += 1

    queue = deque(v for v in range(n) if in_degree[v] == 0)
    topo_order = []

    while queue:
        u = queue.popleft()
        topo_order.append(u)
        for v in graph.get(u, []):
            in_degree[v] -= 1
            if in_degree[v] == 0:
                queue.append(v)

    if len(topo_order) != n:
        return None # cycle detected
    return topo_order

# Example: 5 -> 2 -> 3 -> 1; 4 -> 0 -> 1
graph = {5:[2,0], 4:[0,1], 2:[3], 3:[1], 0:[1], 1:[]}
print(kahn_topological_sort(graph, 6))
# e.g. [4, 5, 0, 2, 3, 1]

```

Comparison: Kahn's vs DFS-based Topological Sort

Feature	Kahn's Algorithm	DFS-based
Approach	BFS (iterative)	DFS (recursive)
Cycle detection	Built-in (result length)	Needs extra logic
Stack overflow risk	None	Possible on large graphs
Order type	Lexicographically smallest (with min-heap)	Reverse post-order

Applications

- Build systems and task schedulers (make, Gradle, Maven).
- Package dependency resolution (npm, pip, apt).
- Course prerequisite ordering in academic planning.

- Instruction scheduling in compilers.
- Spreadsheet formula evaluation order.

Complexity

Metric	Value
Time complexity	$O(V + E)$
Space complexity	$O(V + E)$

РУССКИЙ

Что такое топологическая сортировка?

Топологическая сортировка направленного ациклического графа (DAG) — это линейное упорядочение вершин, при котором для каждого ребра (u, v) вершина u стоит перед вершиной v . Топологическая сортировка возможна только для DAG — графа без направленных циклов.

Алгоритм Кана реализует топологическую сортировку итеративным BFS-подходом, последовательно удаляя вершины с нулевой входящей степенью.

Понятие входящей степени

Входящая степень вершины — это количество направленных рёбер, входящих в неё. Вершина с нулевой входящей степенью не имеет предшественников и может быть обработана первой.

Пошаговый алгоритм

1. Вычислить входящую степень каждой вершины.
2. Добавить в очередь все вершины с нулевой входящей степенью.
3. Пока очередь не пуста: извлечь вершину u , добавить в результат; уменьшить входящую степень соседей; добавить в очередь те, у кого степень стала 0.
4. Если длина результата меньше V — в графе есть цикл.

Реализация на Python

```

from collections import deque

def kahn_topological_sort(graph, n):
    in_degree = [0] * n
    for u in graph:
        for v in graph[u]:
            in_degree[v] += 1
    queue = deque(v for v in range(n) if in_degree[v] == 0)
    topo_order = []
    while queue:
        u = queue.popleft()
        topo_order.append(u)
        for v in graph.get(u, []):
            in_degree[v] -= 1
            if in_degree[v] == 0:
                queue.append(v)
    if len(topo_order) != n:
        return None # цикл обнаружен
    return topo_order

```

Применения

- Системы сборки и планировщики задач (make, Gradle, Maven).
- Разрешение зависимостей пакетов (npm, pip, apt).
- Упорядочение курсов с предварительными требованиями.
- Планирование инструкций в компиляторах.
- Порядок вычисления формул в электронных таблицах.

Сложность

Метрика	Значение
Временная сложность	$O(V + E)$
Пространственная сложность	$O(V + E)$

ՀԱՅԵՐԵՆ

Ի՞նչ է տոպոլոգիական դասավորությունը

Ուղղորդված ացիկլային գրաֆի (DAG) **տոպոլոգիական դասավորությունը** գրաֆի գագաթների գծային կարգ է, որտեղ ամեն կողի (u, v) համար u-ն գալիս է v-ից առաջ: Տոպոլոգիական դասավորությունը հնարավոր է **միայն** DAG-ում (ուղղորդված ցիկլ-չունեցող գրաֆ):

Կանի ալգորիթմը BFS-ի վրա հիմնված կրկնվող մոտեցմամբ իրականացնում է տոպոլոգիական դասավորություն՝ հաջորդաբար հեռացնելով գրոյական in-degree ունեցող գագաթները:

In-degree հասկացություն

Գագաթի **in-degree**-ն դեպի նա ուղղված կողերի քանակն է: Զրոյական in-degree ունեցող գագաթն ամբողջ նախապայման չունի և կարող է առաջինը մշակվել:

Python իրականացում

```
from collections import deque

def kahn_topological_sort(graph, n):
    in_degree = [0] * n
    for u in graph:
        for v in graph[u]:
            in_degree[v] += 1
    queue = deque(v for v in range(n) if in_degree[v] == 0)
    topo_order = []
    while queue:
        u = queue.popleft()
        topo_order.append(u)
        for v in graph.get(u, []):
            in_degree[v] -= 1
            if in_degree[v] == 0:
                queue.append(v)
    if len(topo_order) != n:
        return None # ցիկլ հայտնաբերված
    return topo_order
```

Կիրառություններ

- Կառուցման համակարգեր և խնդիրների պլանավորիչներ (make, Gradle, Maven):
- Փաթեթների կախվածության լուծում (npm, pip, apt):
- Կուրսերի կարգ ուսումնական ծրագրերում:
- Կոմպիլյատորներում հրահանգների ժամանակացույց:

Բարդություն

Kruskal's Algorithm

Category: Minimum Spanning Tree (MST)

ENGLISH

What is a Minimum Spanning Tree?

A **Spanning Tree** of a connected, undirected graph is a subgraph that includes all vertices and is a tree (connected, acyclic). The **Minimum Spanning Tree (MST)** is the spanning tree whose total edge weight is the smallest possible among all spanning trees. MSTs are not necessarily unique — multiple MSTs can exist if some edge weights are equal.

Kruskal's Algorithm, published by Joseph Kruskal in 1956, builds the MST by greedily selecting edges in increasing order of weight, skipping any edge that would form a cycle.

Step-by-Step Algorithm

1. Sort all edges by weight in non-decreasing order.
2. Initialize a Union-Find (DSU) structure with each vertex as its own set.
3. For each edge (u, v, w) in sorted order:
 - If u and v are in different components ($\text{find}(u) \neq \text{find}(v)$): add the edge to the MST and union the two components.
 - Otherwise, skip the edge (it would create a cycle).
4. Stop when $V-1$ edges have been added (MST is complete).

Python Implementation


```

class DSU:
    def __init__(self, n):
        self.parent = list(range(n))
        self.rank = [0] * n

    def find(self, x):
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])
        return self.parent[x]

    def union(self, x, y):
        rx, ry = self.find(x), self.find(y)
        if rx == ry:
            return False
        if self.rank[rx] < self.rank[ry]:
            rx, ry = ry, rx
        self.parent[ry] = rx
        if self.rank[rx] == self.rank[ry]:
            self.rank[rx] += 1
        return True

def kruskal(n, edges):
    # edges: list of (weight, u, v)
    edges.sort()
    dsu = DSU(n)
    mst = []
    total_weight = 0
    for w, u, v in edges:
        if dsu.union(u, v):
            mst.append((u, v, w))
            total_weight += w
    return mst, total_weight

# Example (0-indexed vertices)
edges = [(4,0,1),(8,0,7),(11,1,7),(8,1,2),(7,7,8),(1,7,6),
         (2,8,6),(6,8,2),(4,2,5),(2,2,3),(14,5,3),(9,3,4),(10,5,4),(7,1,2)]
mst, cost = kruskal(9, edges)
print(f"MST cost: {cost}") # 37

```

Why Does Kruskal's Work? (Greedy Correctness)

Kruskal's algorithm is correct because of the *Cut Property*: for any cut of the graph, the minimum-weight crossing edge is guaranteed to be in some MST. By always choosing the lightest safe edge (one that doesn't form a cycle), the algorithm constructs a globally optimal spanning tree.

Kruskal's vs Prim's Algorithm

Feature	Kruskal's	Prim's
Approach	Edge-based (global sort)	Vertex-based (grow from one vertex)
Best for	Sparse graphs	Dense graphs
Data structure	Union-Find	Priority queue
Time complexity	$O(E \log E)$	$O(E \log V)$ with binary heap

Applications

- Designing minimum-cost networks (power lines, pipelines, telecommunications).
- Cluster analysis in machine learning (single-linkage clustering).
- Circuit design: minimizing wire length on a board.
- Network design for computer networks and road systems.

Complexity

Metric	Value	Note
Time complexity	$O(E \log E)$	Dominated by edge sorting
Space complexity	$O(V + E)$	DSU + edge list

РУССКИЙ

Что такое минимальное остовное дерево?

Остовное дерево связного неориентированного графа — подграф, включающий все вершины и являющийся деревом (связный, без циклов). **Минимальное остовное дерево (MST)** — это остовное дерево с наименьшей суммарной стоимостью рёбер среди всех возможных остовных деревьев.

Алгоритм Краскала, опубликованный Джозефом Краскалом в 1956 году, строит MST жадным методом: рёбра добавляются в порядке возрастания веса, пропуская те, что создают цикл.

Пошаговый алгоритм

1. Отсортировать все рёбра по весу в неубывающем порядке.
2. Инициализировать структуру Union-Find: каждая вершина — отдельное множество.
3. Для каждого ребра (u, v, w) в отсортированном порядке: если u и v в разных компонентах — добавить ребро в MST и объединить компоненты; иначе — пропустить.
4. Остановиться, когда добавлено $V-1$ рёбер.

Реализация на Python

```
class DSU:
    def __init__(self, n):
        self.parent = list(range(n))
        self.rank = [0] * n

    def find(self, x):
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])
        return self.parent[x]

    def union(self, x, y):
        rx, ry = self.find(x), self.find(y)
        if rx == ry:
            return False
        if self.rank[rx] < self.rank[ry]:
            rx, ry = ry, rx
        self.parent[ry] = rx
        if self.rank[rx] == self.rank[ry]:
            self.rank[rx] += 1
        return True

def kruskal(n, edges):
    edges.sort()
    dsu = DSU(n)
    mst = []
    total_weight = 0
    for w, u, v in edges:
        if dsu.union(u, v):
            mst.append((u, v, w))
            total_weight += w
    return mst, total_weight
```

Применения

- Проектирование сетей минимальной стоимости (ЛЭП, трубопроводы, телекоммуникации).

- Кластерный анализ в машинном обучении (single-linkage clustering).
- Проектирование схем: минимизация длины проводников.
- Оптимизация инфраструктурных сетей.

Сложность

Метрика	Значение	Примечание
Временная сложность	$O(E \log E)$	Определяется сортировкой рёбер
Пространственная сложность	$O(V + E)$	DSU + список рёбер

ՀԱՅԵՐԵՆ

Ի՞նչ է նվազագույն ծածկող ծառը

Կապակցված ոչ ուղղորդված գրաֆի **ծածկող ծառը** ենթագրաֆ է, որն ընդգրկում է բոլոր գագաթները և հանդիսանում է ծառ (կապակցված, ցիկլ-անագաստ):

Նվազագույն ծածկող ծառ (MST)ն այն ծածկող ծառն է, որի կողերի ընդհանուր կշիռն ամենափոքրն է:

Կրուսկալի ալգորիթմը, հրատարակված Ջոզեֆ Կրուսկալի կողմից 1956 թ.-ին, կառուցում է MST-ն ագահ կերպով. կողերն ավելացվում են ըստ կշռի աճման կարգի, բաց թողնելով ցիկլ ձևավորող կողերը:

Քայլ-առ-քայլ ալգորիթմ

- Դասավորել բոլոր կողերն ըստ կշռի (աճման կարգ):
- Ամաչացնել Union-Find կառուցվածք. ամեն գագաթ — առանձին բազմություն:
- Ամեն կողի (u, v, w) համար դասավորված կարգով. եթե u և v տարբեր բաղադրիչներում են — ավելացնել MST-ում և միաձուլել: հակառակ դեպքում — անտեսել:
- Կանգ առնել, երբ $V-1$ կող ավելացված է:

Python իրականացում

```

class DSU:
    def __init__(self, n):
        self.parent = list(range(n))
        self.rank = [0] * n

    def find(self, x):
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])
        return self.parent[x]

    def union(self, x, y):
        rx, ry = self.find(x), self.find(y)
        if rx == ry:
            return False
        if self.rank[rx] < self.rank[ry]:
            rx, ry = ry, rx
        self.parent[ry] = rx
        if self.rank[rx] == self.rank[ry]:
            self.rank[rx] += 1
        return True

def kruskal(n, edges):
    edges.sort()
    dsu = DSU(n)
    mst, total_weight = [], 0
    for w, u, v in edges:
        if dsu.union(u, v):
            mst.append((u, v, w))
            total_weight += w
    return mst, total_weight

```

Կիրառություններ

- Նվազագույն արժեքի ցանցերի նախագծում (Էլ. հաղորդալարեր, խողովակեր, հեռահաղորդակցություն):
- Մեզենայական սուսման կլաստերային միավորում (single-linkage clustering):
- Տեղանքային նախագիծ. հաղորդակցության կառուցում:
- Էնթալպիայի օպտիմալացում:

Բարդություն

Չափանիշ	Արժեք	Նշում
Ժամանակային բարդություն	$O(E \log E)$	Կողերի դասավորությամբ պայմանավորված

Հիշողության բարդություն

$O(V + E)$

DSU + կողերի ցանակ

Graph Algorithms — Educational Reference | English • Russian • Armenian

BFS • Dijkstra • Cycle Detection • Connected Components • Kahn's Algorithm • Kruskal's Algorithm